

Predicting Real-Time Win Probability in League of Legends

Background

This project builds a model that estimates, at any point during a League of Legends match, the probability that a given team will win. League of Legends is a five-versus-five competitive online game in which two teams destroy each other's base while accumulating gold and experience, taking objectives such as towers, dragons, Rift Heralds, and Barons, and fighting for map control. Given the state of a game at a particular minute, a natural question for both spectators and players reviewing their own games is: what is each team's probability of winning from here?

Match data was collected from the Riot Games API in two stages, mirroring how the API itself is organized. Ranked match IDs were first gathered across a spread of skill tiers so the dataset would not be biased toward a single rank bracket, and then, for each match, two API resources were pulled: the match summary, which gives final participant stats and the team result, and the match timeline, which reports the full game state, including gold, experience, kills, objectives, and tower status per participant, at one-minute intervals from the start of the match until it ends.

The project treats win probability as a supervised binary classification problem. Minute-level snapshots of match state are converted into a feature table, and two different model families are trained and compared on an identical held-out test set of 1,955 matches: a gradient-boosted decision tree ensemble (XGBoost) that treats each in-game minute as an independent observation, and a masked-sequence long short-term memory network (LSTM) that models a full match as a time series. Both models are trained on 510 engineered features spanning gold, experience, kill, tower, and objective differentials between the two teams, and both are evaluated on an identical match-level train, validation, and test split so their results are directly comparable.

Problem Statement

Win probability is not directly observable in the data the way a kill or a gold total is; it has to be inferred from everything a game state implies about which team is currently ahead. A team's position in a match can shift quickly through objective trades, skirmishes, or a single well-timed engagement, which makes win probability inherently a moving target rather than a single number attached to a team.

The core question this project asks is whether that moving target can be estimated well using only minute-level game state, and whether an architecture that natively models a match as a sequence (the LSTM) meaningfully outperforms one that treats each minute as an independent row and instead relies on hand-engineered short-horizon momentum features (the XGBoost model). Both models are trained on the same features where possible and evaluated on the same held-out matches, so any gap between them can be attributed to the modeling approach rather than to differences in data.

Pipeline

The project is organized as an automated data pipeline that transforms raw Riot API responses into a trained, evaluated win-probability model. Because the Riot API is rate-limited, collection is a slow, sequential process by design, so it is treated as a one-time, manually triggered step (`collect_data.py`) rather than part of the pipeline that reruns everything downstream. During review, one correctness issue was found and fixed in this stage: an output-directory path in `collect_data.py` no longer matched the folder actually used downstream. A performance issue was fixed alongside it, rewriting the per-frame cumulative event counters used to reconstruct participant deaths and respawns from an $O(\text{frames} \times \text{events})$ rescan into a single $O(\text{events})$ pass over a sorted event log with a monotonic pointer. Both fixes were validated against a synthetic, randomized match before being accepted, without needing to re-run the live, rate-limited collection itself.

Per-rank snapshots are merged into a single dataset (`combine.py`) before feature engineering begins. The resulting held-out test set alone spans 1,955 matches, drawn from a larger combined train, validation, and test pool, with each match contributing one row per in-game minute up to that match's actual length. From there, the pipeline runs feature engineering, a hyperparameter grid search and training pass for each of the two models, and a final evaluation and plotting stage that produces the calibration curves, minute-bucket tables, and feature-importance rankings discussed below.

Feature Engineering

Each minute-level row is converted from raw per-player, per-team counters into a symmetric set of team-versus-team differential features, on the reasoning that win probability should depend on how far ahead or behind a team is, not on the raw magnitude of either team's statistics alone. Basic economy and combat features cover gold, experience, level, kills, assists, and minion or monster kills, expressed as team totals, per-role splits, and 100-minus-200 differentials. Damage statistics record physical, magic, and true damage dealt to champions per role, while objective-control features cover dragons, Rift Herald, Barons, and elder dragons taken, along with outer, inner, and base tower destruction by lane.

Per-minute rate features, such as `gold_diff_per_min` and `xp_diff_per_min`, normalize cumulative leads by game time so that a lead is comparable whether it was built over five minutes or twenty-five. A further set of short-horizon momentum features captures one-minute and three-minute deltas of the key differential statistics, intended to let the tree-based model see recent trends without needing a recurrent architecture. Together these groups produce 510 features per row. Two independent feature-engineering scripts, one for each model, originally duplicated the same role, team, stat, and objective constant lists and the same monotonic-constraint-relevant naming logic; this was consolidated into a shared `common.py` module during a later polish pass so both pipelines stay in sync by construction and matching bugs only need to be fixed once.

The LSTM pipeline additionally reshapes the flat per-minute rows into per-match sequences, padded and masked to the length of the longest match in a batch. The original implementation re-filtered the full dataframe by match ID for every match to build these sequences; it was rewritten to do a single group-by pass up front, producing dictionaries keyed by match ID that are then sliced per split, a straightforward but meaningful performance improvement that scales with the number of matches rather than matches multiplied by dataframe size.

Modeling Methodology

Both models are evaluated on a strictly identical basis. An 80/20 match-level train/test split is created once, rather than per-minute, to avoid leaking a single match's other minutes across the split, and is persisted to a shared file that both training scripts reuse rather than each drawing its own random split, which is what makes the head-to-head comparison valid. Within the training portion, a further 80/20 split, using the same random seed, is carved out in memory as a validation set for early stopping and hyperparameter selection, independently for each model.

XGBoost

XGBoost is trained as a binary classifier, using a binary-logistic objective and the histogram tree method, treating every match-minute pair as an independent row. A grid search over 27 configurations, varying max depth over {3, 4, 6}, learning rate over {0.03, 0.05, 0.1}, and subsample over {0.7, 0.85, 1.0}, with column subsampling fixed at 0.85, is run with early stopping (30 rounds, capped at 600 boosting rounds) against the validation split, selecting the configuration with the lowest validation log-loss. Monotonic constraints are applied automatically based on feature naming convention, so that features representing a “team 200 minus team 100” style differential are constrained to move win probability in the opposite direction from a “team 100” style differential, meaning the model cannot learn, for example, that more red-side gold increases blue-side win probability.

LSTM

The LSTM treats each match as a full sequence and predicts a win probability at every minute simultaneously, using masking so that padding added to align variable-length matches within a batch does not contribute to the loss. A grid search over 24 configurations, varying hidden size over {64, 128}, number of layers over {1, 2}, dropout over {0.1, 0.25, 0.35}, and weight decay over {0, 1e-4}, is run with early stopping on validation log-loss, selecting the best configuration the same way as XGBoost.

Evaluation

Both models are scored on the shared test set using ROC-AUC, log-loss, accuracy at a 0.5 threshold, and Brier score, both overall and broken out by seven minute buckets, from 1–5 through 31–45, to see how each model's discrimination evolves over the course of a match. Calibration is additionally assessed with reliability diagrams that compare predicted probability against observed win rate, binned into deciles.

Results and Analysis

Hyperparameter Search

The XGBoost grid search selected max_depth = 6, learning_rate = 0.03, and subsample = 1.0, with colsample_bytree fixed at 0.85, stopping at boosting round 266 of a possible 600, with a validation AUC of 0.815 and a validation log-loss of 0.519. Across all 27 configurations, validation AUC ranged narrowly from about 0.8144 to 0.8155, so the model is not especially sensitive to these particular hyperparameters within the ranges searched, though the selected configuration is a consistent, if modest, best.

The LSTM grid search selected hidden_size = 128, num_layers = 2, dropout = 0.1, and weight_decay = 1e-4, converging after only three epochs, with a validation AUC of 0.811 and a validation log-loss of 0.524. As with

XGBoost, results across the 24 configurations were tightly clustered, with validation AUC roughly between 0.808 and 0.812, suggesting the architecture is fairly robust to these choices and that data and features, rather than model capacity, are the binding constraint on performance. Total pipeline run time end to end, including both grid searches, was 13.3 minutes: about two minutes for combining per-rank snapshots, under a minute combined for feature engineering on both pipelines, roughly 7.7 minutes for the 27-configuration XGBoost grid search, and roughly 2.4 minutes for the 24-configuration LSTM grid search, with the remaining time spent on summary statistics and plotting.

Overall Test-Set Performance

On the shared 1,955-match test set, XGBoost and LSTM perform comparably, with XGBoost holding a small edge on every metric. Across 57,833 minute-level rows, XGBoost reaches a test-set ROC-AUC of 0.808, a log-loss of 0.527, an accuracy of 71.8%, and a Brier score of 0.179. Across 57,591 minute-level rows, the LSTM reaches an AUC of 0.803, a log-loss of 0.532, an accuracy of 71.4%, and a Brier score of 0.181. The gap is small enough that the two approaches should be read as roughly equally capable at this task rather than one clearly superior to the other; XGBoost's advantage may partly reflect that its one-minute and three-minute momentum features hand-engineer some of the temporal signal the LSTM has to learn on its own from far less data per match than a row-independent model effectively sees.

Performance by Minute Bucket

Both models follow the same, intuitive trajectory: discrimination is weak in the opening minutes, before either team has built a meaningful lead, and improves steadily as the game develops, before a modest late-game dip. XGBoost's AUC rises from 0.615 in minutes 1–5 to a peak of 0.906 in minutes 26–30, with accuracy climbing from 57.7% to 82.4% over the same range, before easing to an AUC of 0.867 and an accuracy of 78.7% in the 31–45 bucket. The LSTM follows an almost identical shape, from an AUC of 0.584 and 55.8% accuracy in minutes 1–5 up to an AUC of 0.901 and 81.1% accuracy in minutes 26–30, before settling at an AUC of 0.875 and an accuracy of 79.5% in minutes 31–45.

This late-game dip was investigated as a potential defect but is better explained by two compounding, non-modeling factors. First, sample size shrinks substantially in that bucket; only about 36.7% of matches, roughly 4,800 to 5,700 of the more than 57,000 rows depending on the model, reach minute 31 at all, since most games end between minutes 25 and 35, so the 31–45 estimate is inherently noisier than earlier buckets. Second, and more fundamentally, games that are still undecided after minute 30 are disproportionately close games; a match that was already lopsided by minute 25 typically ends before minute 31, so the population of matches remaining in the 31–45 bucket is selected precisely for being harder to call. The dip is therefore a property of which games survive to be measured, not a sign that either model degrades with more information.

Calibration

Reliability diagrams for both models show predicted probabilities tracking observed win rates closely across deciles, with XGBoost's Brier score of 0.1787 and the LSTM's 0.1806 essentially tied, and both meaningfully better than a coin-flip baseline, whose Brier score would be 0.25, or the majority-class baseline.

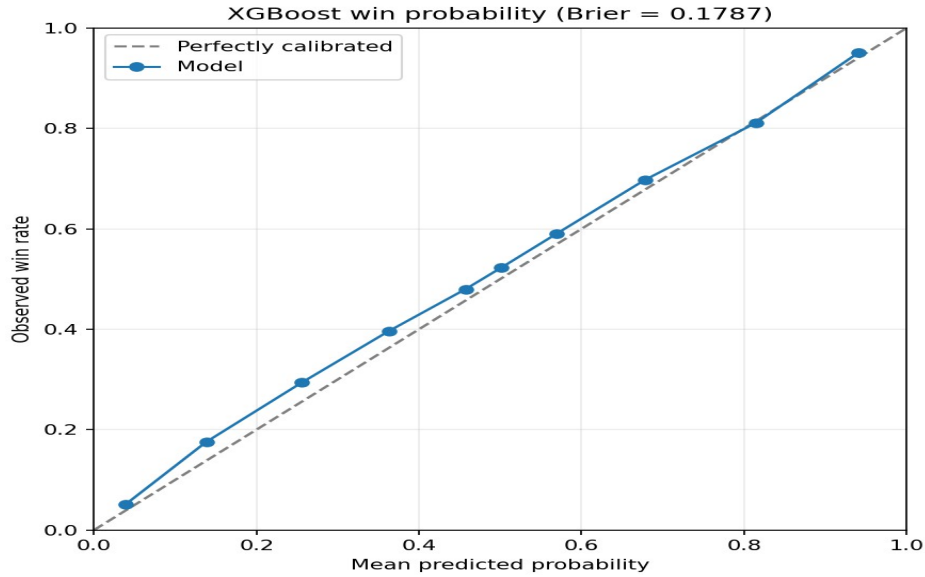


Figure 1. XGBoost calibration curve (reliability diagram), test set.

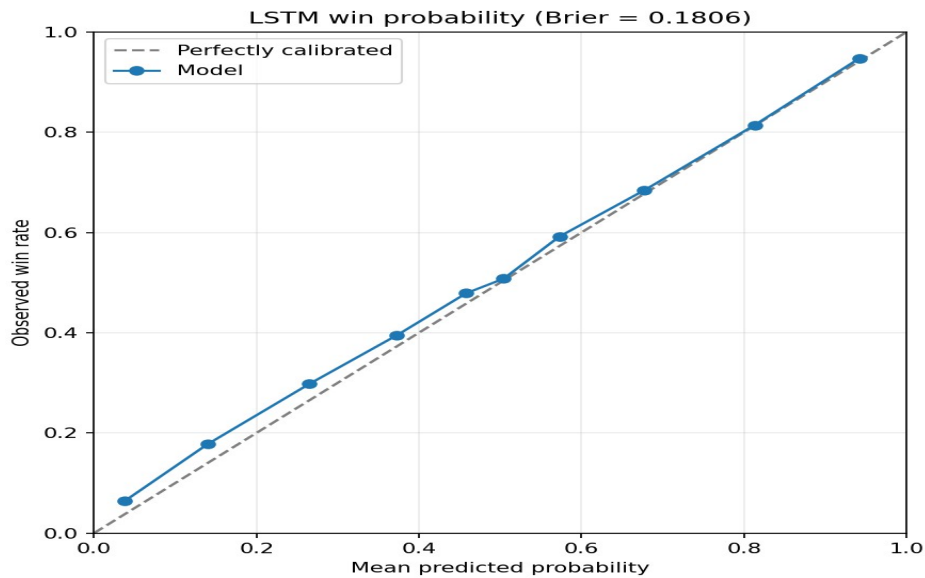


Figure 2. LSTM calibration curve (reliability diagram), test set.

Feature Importance

XGBoost's built-in gain-based feature importance, the average loss reduction attributed to a feature across all splits that use it, was computed for all 510 features. Of these, 362 were used in at least one split, while the remaining 148 contributed nothing to this particular trained model. Gold and experience differentials dominate: the single feature `team_total_gold_diff` accounts for 22.5% of total gain on its own, and the top ten features together account for about 51% of total gain. The next-highest-ranked features are `team_100_gold_share` (7.8%), `gold_diff_per_min` (6.8%), `xp_diff_per_min` (3.8%), `team_xp_diff` (2.7%), `dragons_diff` (2.3%), the three-minute delta of `team_total_gold_diff` (1.8%), `team_level_diff` (1.2%), and the three-minute deltas of `team_kills_diff` and `team_xp_diff` (1.1% and 0.9%, respectively).

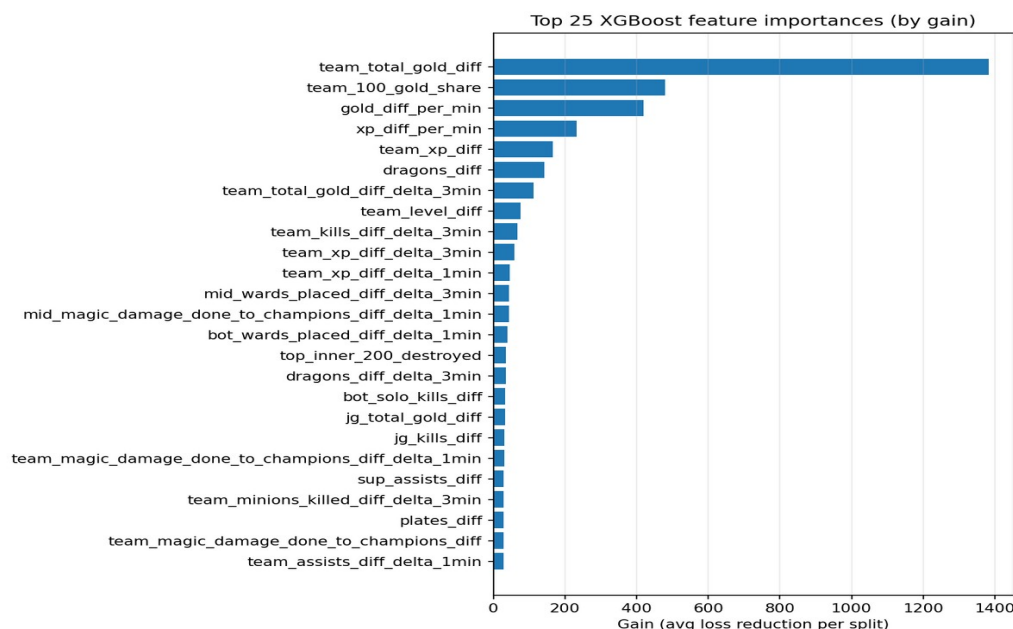


Figure 3. Top 25 XGBoost features by gain.

This ranking is consistent with domain intuition, since gold and experience differentials are the most direct proxies for which team is currently stronger, and with an observation made while building the interactive predictor described below: at a synthetic, out-of-distribution game state where gold and experience are held exactly even, changing objectives, towers, or first blood alone moves the predicted probability only slightly, because the model has learned that those signals matter mostly through the economic and level leads they eventually produce, not as standalone predictors.

Feature Redundancy

Gain-based importance shows which signal a tree preferred when every feature was available, but it cannot distinguish a feature that matters from one that is simply redundant with a cleaner feature — and since almost every event in League of Legends, a kill, a tower, an objective, eventually shows up as a gold or experience lead, it is worth testing directly whether the non-economic features carry any independent predictive information at all. Both models were retrained with every gold, experience, and level feature removed — 81 of 510 features for XGBoost and 26 of 114 for the LSTM — reusing the same shared train, validation, and test split and the same hyperparameters selected by the grid searches above, so that any change in performance is attributable to the missing features rather than to a different split or a different configuration. A second ablation repeats the same procedure for objective features instead: dragons, Rift Heralds, Barons, elder dragons, plates, and every tower destroyed counter, 95 of 510 features for XGBoost and 31 of 114 for the LSTM.

The economy ablation shows a substantially smaller drop than the feature-importance ranking alone would suggest. XGBoost's test AUC falls only from 0.8074 to 0.8052, a loss of 0.0022, retaining 99.7% of its discrimination using towers, objectives, kills, and their one- and three-minute momentum deltas alone. The LSTM is essentially unaffected, moving from a test AUC of 0.8011 to 0.8012 with the economy features removed — no measurable loss at all. This indicates that the feature-importance ranking in the previous section reflects which signal the model preferred, not which signal it needed: kills, towers, and objectives are what produce a

gold and experience lead in the first place, so a model given a sufficiently rich non-economic description of the game state can reconstruct nearly all of the same information gold represents directly, just less efficiently.

The objective ablation tells a different story. Removing dragons, Heralds, Barons, elders, and towers costs XGBoost an AUC of 0.0095 (0.8074 to 0.7979, retaining 98.8%) and costs the LSTM an AUC of 0.0108 (0.8011 to 0.7903, retaining 98.7%) — roughly four to five times the damage done by removing the much larger economy feature group. Both models lose more accuracy from a smaller set of missing features, which suggests objective control is not fully substitutable the way gold and experience are: it carries some independent signal, beyond what it produces in gold and experience, that per-role damage, kill, and momentum features alone do not fully replicate. A plausible reading is that objectives encode map-control and tempo information, such as vision, jungle pressure, and macro positioning, that only partially shows up as a numeric economic lead by the time a given minute snapshot is taken.

Match Replay Example

As a qualitative check, predicted win probability over time was plotted for a sample of test matches, overlaid with tower and objective events. In the example below, from match NA1_5488849653, won by the blue side, both models track the match's swings in a broadly similar shape, with the XGBoost curve reacting slightly more sharply to individual events, consistent with its explicit momentum-delta features, and the LSTM curve noticeably smoother.

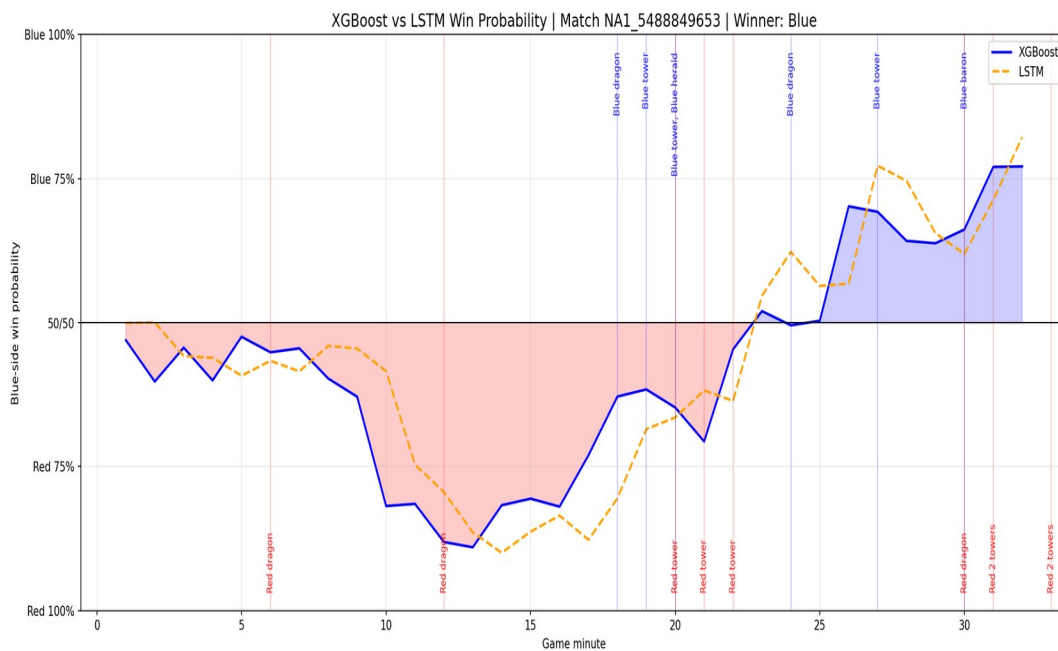


Figure 4. XGBoost vs. LSTM win-probability trajectory for a sample test match, with tower and objective events annotated.

Interactive Application

Alongside this report, the project includes a three-page Streamlit application that makes the pipeline's outputs explorable without rerunning any code. A match replay explorer reproduces the trajectory chart above interactively for any test match, with hover tooltips and event annotations. A model performance dashboard

surfaces calibration curves, minute-bucket comparison tables, feature-importance charts, both grid-search result tables, and pipeline run times, all read live from the pipeline's saved output files. A live what-if predictor lets a user set gold, experience, CS, and kill differentials, tower and objective counts, and first blood, and see the trained XGBoost model's predicted win probability update live through a gauge chart; a callout in the app explains the gold and experience dominance finding above so the tool's behavior at extreme, unrealistic inputs, such as objectives changing while gold is held at zero, is not mistaken for a bug. The app is designed to degrade gracefully, so that pages depending on optional local artifacts, such as the full per-minute dataset for event annotations or a freshly generated feature-importance file, fall back to an informative message rather than crashing if those files are not present in a given checkout.

Discussion

The central empirical result is that a carefully feature-engineered, row-independent gradient-boosted tree model and a sequence-aware recurrent network reach almost identical predictive performance on this task, with an AUC of 0.808 versus 0.803. This suggests that most of the temporal signal useful for win prediction in League of Legends can be captured through explicit short-horizon momentum features, the one-minute and three-minute deltas, without needing a model that natively processes sequences, at least at the one-minute granularity used here. A model with access to finer-grained event timing, or one trained to predict further into the future rather than the current instant, might see a larger gap between the two approaches.

The feature-importance results and the what-if predictor's behavior together tell a consistent story: gold and experience leads are the dominant, almost sufficient signal for win probability, and other observable game state, such as towers, objectives, and first blood, matters mainly insofar as it produces those economic leads. The feature-redundancy ablation confirms this is largely a collinearity effect rather than the non-economic features being uninformative on their own: removing every gold, experience, and level feature costs XGBoost only 0.002 AUC and costs the LSTM nothing measurable, showing that objectives, towers, and kills already carry almost all the same information gold represents, just less directly. The reverse ablation, removing objectives instead, costs both models four to five times as much AUC, which suggests objective control carries some genuinely independent signal that the economy features do not fully substitute for — a nuance the raw feature-importance ranking, dominated by gold as it is, would not surface on its own. This is a reasonable overall finding for a game where gold funds items that are a primary lever on combat power, but it also means a model like this one is not well suited to answering counterfactual questions, such as how much a single objective mattered on its own, without also holding constant the resulting economic swing.

The late-game accuracy dip is a useful reminder that per-bucket metrics on a shrinking, non-random subsample should not be read the same way as metrics on a stable population; the games that reach minute 31 are a selected group, and their harder-to-call nature is itself informative about the game, not primarily a weakness of either model.

Conclusion and Future Work

This project delivers an end-to-end pipeline for predicting League of Legends win probability from Riot API match data, two independently tuned models that reach comparable, well-calibrated performance, with AUC between about 0.80 and 0.81 and Brier score around 0.18, and an interactive application that makes the results

and the model itself explorable. Along the way, the pipeline was substantially hardened: a shared `common.py` module removed duplicated feature and evaluation logic across four scripts, both training scripts gained early stopping, monotonic constraints for XGBoost, and hyperparameter grid search, a stale-file bug in the plotting script was fixed, and XGBoost feature importances are now saved automatically as part of training. Two feature-redundancy ablations were also added (`ablation.py`), retraining both models once with all gold, experience, and level features removed and once with all objective features removed, to test directly whether the game's other observable state carries independent predictive value. Removing the economy features costs almost nothing, but removing objectives costs four to five times more, showing the two feature groups are not equally substitutable.

Several natural extensions were not pursued here. One would be evaluating a patch-level, or game-version, holdout split in addition to the current random match split, to test how well the model generalizes across balance patches rather than just across matches drawn from the same patch mix. Another would be adding permutation importance for the LSTM, to get a feature-importance view for the sequence model comparable to XGBoost's gain-based ranking. A third would be extending the momentum features to longer horizons, such as five-minute or ten-minute deltas, to see whether the LSTM's relative advantage over XGBoost grows once explicit short-horizon features are removed, which would more directly test how much of the LSTM's value is really about sequence modeling versus what hand-engineered features already capture. A fourth would be training a version of the LSTM to predict a fixed number of minutes into the future rather than the current instant, arguably the more interesting product use case of forecasting what will happen rather than describing what is happening, and one where a sequence model's advantage over a row-independent one should be more pronounced.

References

- [1] Riot Games. (n.d.). Riot Games API. Retrieved August 2026, from <https://developer.riotgames.com/>
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [3] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [5] Streamlit Inc. (n.d.). Streamlit documentation. Retrieved August 2026, from <https://docs.streamlit.io/>